

Lab1: DataLab

睦罗涛 25303090064

一、实验目标

在限制整数位运算函数个数和常量范围的情况下，完成 P1-P19 任务，在学习二进制的同时，加深对位运算的理解。

二、实验内容

P1 signMask

即返回一个只有最高位为 1 的掩码

```
int signMask(void)
{
    return 1<<31;
}
```

P2 bitXor

异或的普通形式为

$$x^y = (x \& \sim y) | (\sim x \& y)$$

但题目不允许|，所以用德摩根定律得到：

$$x^y = \sim((x \& y) | (\sim x \& \sim y)) = \sim(x \& y) \& \sim(\sim x \& \sim y)$$

```
int bitXor(int x, int y)
{
    return ~(x&y)&~(~x&~y);
}
```

P3 negativePart

题目要求是若 $x < 0$ ，返回 $-x$ ；否则返回 0，且不能写 $-x$ 。故采用符号掩码：mask = $x \gg 31$ 当 $x \geq 0$ 时 mask = 0，结果为 0；当 $x < 0$ 时 mask = -1，结果等于 $\sim x + 1$

```
int negativePart(int x)
{
    int mask=x>>31;
    return mask&(~x+1);
}
```

P4 copyByteWithin

即把 32 位整数 x 的第 src 字节复制到第 dst 字节，其他字节保持不变。

字节编号要换成 bit 位移

$$\text{shift} = \text{byteIndex} * 8 = \text{byteIndex} \ll 3$$

整个过程分三步：

取出源字节：byteVal = $(x \gg \text{srcShift}) \& 0xFF$

清掉目标字节原来内容：mask = $\sim(0xFF \ll \text{dstShift})$

把源字节移到目标位置并写回： $(x \& \text{mask}) | (\text{byteVal} \ll \text{dstShift})$

```
int copyByteWithin(int x, int src, int dst)
{
    int srcShift = src << 3;
    int dstShift = dst << 3;
    int byteVal = (x >> srcShift) & 0xFF;
    int mask = ~(0xFF << dstShift);
    return (x & mask) | (byteVal << dstShift);
}
```

P5 logicalShift

即把 x 逻辑右移 n 位，高位补 0。

C 语言中 $x \gg n$ 对 int 是算术右移：非负数：高位补 0，正好就是逻辑右移；负数：高位补 1，会保留符号扩展。所以核心是做掩码，把算术右移后高 n 位补出来的 1 清掉。希望构造一个掩码满足低 $32 - n$ 位为 1，高 n 位为 0：

$$\text{mask} = \sim((1 \ll 31) \gg n \ll 1)$$

```
int logicalShift(int x, int n)
{
    int arith = x >> n;
    int mask = ~((1 << 31) >> n << 1);
    return arith & mask;
}
```

P6 swapNibblePairs

题目本质对每个字节内部，高低 4 位互换 $0xAB \rightarrow 0xBA$

通过

$$m4 = 0x0F$$

$$\text{lowMask} = m4 | (m4 \ll 8) | (m4 \ll 16) | (m4 \ll 24)$$

构造掩码

$$\text{lowMask} = 0x0F0F0F0F$$

来取每个字节中的低四位。先通过 $x \gg 4$ ，把每个字节的高 nibble 移到低 nibble；再用 lowMask 把每个低 nibble 留下，同时把算术右移产生的符号扩展清掉，最后补上高 nibble 完成任务。

```
int swapNibblePairs(int x)
{
    int m4 = 0x0F;
    int lowMask = m4 | (m4 << 8) | (m4 << 16) | (m4 << 24);
    int low4 = x & lowMask;
    int highToLow = (x >> 4) & lowMask;
    return (low4 << 4) | highToLow;
}
```

P7 secondLowestZeroBit

题目要求为返回一个掩码，标记 x 中“第二低的 0bit”的位置。若没有第二低 0 位，返回 0。由于最低位 0 无法寻找，所以对 x 取反

$$\text{notx} = \sim x$$

x 中的 0 位变成 notx 中的 1 位，于是问题变成找 notx 中第二低的 1 位。取最低 1 位并清除

$$\text{mask1} = \text{notx} \& (\sim \text{notx} + 1)$$

$$\text{rest} = \text{notx} \wedge \text{mask1}$$

再取 rest 的最低 1 位，就是原 x 的次低 0 位。

```
int secondLowestZeroBit(int x)
{
    int notx = ~x;
    int mask1 = notx & (~notx + 1);
    int rest = notx ^ mask1;
    int mask2 = rest & (~rest + 1);
    return mask2;
}
```

P8 oddParity

题目要求为若 x 的二进制中 1 的个数是偶数，返回 1；否则返回 0。考虑到一组 bit 异或起来，结果就是“1 的个数是否为奇数”的奇偶位。不断折叠：

$$x^{\wedge} = x \gg 16$$
$$x^{\wedge} = x \gg 8$$
$$x^{\wedge} = x \gg 4$$
$$x^{\wedge} = x \gg 2$$
$$x^{\wedge} = x \gg 1$$

最后只剩最低位有意义—— $x \& 1 = 1$ ：奇数个 1； $x \& 1 = 0$ ：偶数个 1

```
int oddParity(int x)
{
    x ^= x >> 16;
    x ^= x >> 8;
    x ^= x >> 4;
    x ^= x >> 2;
    x ^= x >> 1;
    return !(x & 1);
}
```

P9 rotateRightBits

把 32 位整数循环右移 n 位、右边 n 位绕到左边可以通过 $x \ll (32 - n)$ 和 $x \gg n$ 实现。但必须逻辑右移，通过构造掩码

$$\text{int mask} = \sim((1 \ll 31) \gg n \ll 1)$$

清掉高 n 位最后相或。

```
int rotateRightBits(int x, int n)
{
    int leftPart = x << (32 + (~n + 1));
    int high = x >> n;
    int mask = ~((1 << 31) >> n << 1);
    int rightPart = high & mask;
    return leftPart | rightPart;
}
```

P10 roundEvenPow2

题目要求为把非负 x 舍入到最近的 2^n 倍数，若正好在两个倍数中间，则选商为偶数的那个倍数。首先通过 $\text{unit} = 2^{n-1}$ 和 $r = (x + \text{half}) \gg n$ 进行判断（ rem 为余数）——

若 $\text{rem} < \text{half}$ ， r 就是下界商 q ；

若 $\text{rem} > \text{half}$ ， r 是上界商 $q+1$ ；

若 $\text{rem} == \text{half}$ ，正好 $\text{tie} == 1$ ， $r = q+1$ 。这时引入题目额外判断条件，若 q 为偶，则 $r = q+1$ 为奇，要退回 q 。若 q 为奇，则 $r = q+1$ 为偶，保留。即：

$$\text{adj} = \text{tie} \& (r \& 1)$$
$$\text{return } (r + (\sim \text{adj} + 1)) \ll n$$

```
int roundEvenPow2(int x, int n)
{
    int mask = (1 << n) + (~0);
    int half = mask ^ (mask >> 1);
    int r = (x + half) >> n;
    int rem = x & mask;
    int tie = !(rem ^ half);
    int adj = tie & (r & 1);
    return (r + (~adj + 1)) << n;
}
```

P11 midpointTowardFirst

题目要求数学中点 $(x+y)/2$,若正好在两个整数中间,则选更靠近第一个参数 x 的那个整数。考虑到 $x+y$ 可能溢出,所以先求 x 和 y 的一半再相加,最后考虑最低位:

$$\text{base} = (x \gg 1) + (y \gg 1) + ((x \oplus y) \& 1)$$

这就是不溢出的 floor 中点。

若 $x+y$ 为偶数,结果就是 base;

若 $x+y$ 为奇数,则需要在 base 和 base+1 中选一个靠近 x 的,根据题目条件,当 $x > y$ 时, x 是较大的端点,中点应向上取到 base+1; 当 $x < y$ 时, x 是较小端点,应保留 base。而判断 x 和 y 大小关系,首先记录 x 和 y 的正负性:

`int sx = x >> 31`

`int sy = y >> 31`

然后分同号和异号两种情况

同号: $\sim(sx \oplus sy) \& ((\sim d) \gg 31)$

异号: $(sx \oplus sy) \& (\sim sx)$

```
int midpointTowardFirst(int x, int y)
{
    int hx = x >> 1;
    int hy = y >> 1;
    int bothOdd = (x & y) & 1;
    int oddSum = (x ^ y) & 1;
    int sx = x >> 31;
    int sy = y >> 31;
    int d = x + (~y + 1);
    int gt = ((~(sx ^ sy) & ((~d) >> 31)) | ((sx ^ sy) & (~sx))) & 1;
    return hx + hy + bothOdd + (oddSum & gt);
}
```

P12 isBetweenEitherOrder

题目本质判断 x 是否在闭区间 $[a,b]$ 内, a,b 可任意顺序。直观上可以通过 $x - a \geq 0$ 和 $b - x \geq 0$, 但在 INT_MIN 附近会溢出。对于减法 $d = x - a$, 没有溢出时, d 的符号就是数学符号; 有溢出时, d 的数学符号等于 x 的符号, 溢出判断为

$$\text{ov} = ((x \wedge a) \& (x \wedge d)) \gg 31$$

从而数学符号 s 为

$$s = (\text{ov} \& (x \gg 31)) | (\sim \text{ov} \& (d \gg 31))$$

对 a,b 分别得到 $s1,s2$: 若 $s1$ 和 $s2$ 异号,说明 $x - a$ 与 $x - b$ 一个非正一个非负, x 在两 endpoint 之间或某端点(通过 $z1$ 和 $z2$ 判断端点)

```
int isBetweenEitherOrder(int x, int a, int b)
{
    int d1 = x + (~a + 1);
    int d2 = x + (~b + 1);
    int ov1 = ((x ^ a) & (x ^ d1)) >> 31;
    int ov2 = ((x ^ b) & (x ^ d2)) >> 31;
    int sx = x >> 31;
    int s1 = (ov1 & sx) | (~ov1 & (d1 >> 31));
    int s2 = (ov2 & sx) | (~ov2 & (d2 >> 31));
    int z1 = !d1;
    int z2 = !d2;
    return (z1 | z2 | (s1 ^ s2)) & 1;
}
```

P13 mul5Sat

题目本质返回 $x * 5$ ，若溢出则饱和到 `INT_MAX` 或 `INT_MIN`。

$x * 5$ 可表示为：

$$5x = 4x + x = (x \ll 2) + x$$

考虑正数溢出条件：

$$\text{floor}(\text{INT_MAX}/5) = 429496729 = 0x19999999 \text{ 即 } x \geq 0x1999999A$$

负数溢出条件：

$$\text{ceil}(\text{INT_MIN}/5) = -429496729 \text{ 即 } x \leq -429496730$$

从而令 $Q = 0x1999999A$ 正溢出为 $x - Q \geq 0$ 且 x 非负；负溢出为 $x + Q \leq 0$ 且 x 为负。

```
int mul5Sat(int x)
{
    int Q = (0x19 << 24) | (0x99 << 16) | (0x99 << 8) | 0x9A;
    int fiveX = (x << 2) + x;
    int signX = x >> 31;
    int diffP = x + (~Q + 1);
    int posT = ~(diffP >> 31);
    int posOv = (~signX) & posT;
    int diffN = x + Q;
    int eq = (!diffN << 31) >> 31;
    int negT = (diffN >> 31) | eq;
    int negOv = signX & negT;
    int overflow = posOv | negOv;
    int sat = (1 << 31) + (~signX);
    return fiveX ^ (overflow & (fiveX ^ sat));
}
```

P14 classifyAdd3

题目本质为判断三个 `int` 的精确数学和是否在 `int` 范围内。由于中间可能溢出后又回绕，所以不能先算 $x + y + z$ 后看符号。那么就需要给每次加法记录“溢出方向”。对 $a + b$ ，只有三种情况：同正相加，结果变负（正溢出）贡献 $+2^{32}$ ；同负相加，结果变非负（负溢出）贡献 -2^{32} ；否则：无溢出，贡献 0。拆分成两次加法后，溢出贡献分别为 sig1 和 sig2 ，所以精确和满足：

$$\text{exact} = s + (\text{sig1} + \text{sig2}) * 2^{32}$$

因此只需看 $m = \text{sig1} + \text{sig2}$ ：

$m > 0$ ： $\text{exact} > \text{INT_MAX}$ ，返回 1；

$m < 0$ ： $\text{exact} < \text{INT_MIN}$ ，返回 -1；

$m = 0$ ： $\text{exact} = s$ ，合法，返回 0。

单次溢出方向的判断可以通过 x ， y 和结果的符号判断

正溢出： $p1 = (\sim sx) \& (\sim sy) \& ss1$ ；负溢出 $n1 = sx \& sy \& (\sim ss1)$

最终贡献 $\text{sig1} = (p1 \& 1) + (n1 \& (\sim 1 + 1))$

```
int classifyAdd3(int x, int y, int z)
{
    int s1 = x + y;
    int sx = x >> 31;
    int sy = y >> 31;
    int ss1 = s1 >> 31;
    int p1 = (~sx) & (~sy) & ss1;
    int n1 = sx & sy & (~ss1);
    int sig1 = (p1 & 1) + (n1 & (~1 + 1));
    int s = s1 + z;
    int sz = z >> 31;
    int ss = s >> 31;
    int p2 = (~ss1) & (~sz) & ss;
    int n2 = ss1 & sz & (~ss);
    int sig2 = (p2 & 1) + (n2 & (~1 + 1));
    int m = sig1 + sig2;
    int nz = !m;
    int pos = ((m >> 31) + 1) & nz;
    return pos + (m >> 31);
}
```

P15 floatScaleThreeHalves

题目要返回 $f * 3/2$ 的单精度位模式，要求 round - to - nearest - even，保留+0/-0 符号，NaN 原样返回。按照阶码进行分类：

exp == 0xFF: NaN 或 Inf，直接返回；

exp == 0: 非规格化，没有隐藏 1；

其余情况：规格化，要补隐藏位 0x800000。

对于非规格化分支，通过

$$t = \text{frac} + (\text{frac} \gg 1)$$

实现乘法然后进行舍入

实现当精确值为 $t + 0.5$ 时，若 t 为奇，则进到下一个偶数；若 t 为偶，则保留。返回时直接把 t 塞回低 23+位域，因为非规格化乘 1.5 后仍可用同一段位模式解释；若越过 bit23，会自动表现为规格化数。

对于规格化分支，需要先补隐藏位

$$a = \text{frac} | 0x800000$$

乘 1.5 后进行第一次舍入

$$t = a + (a \gg 1)$$

$$r1 = (a \& 1) \& (t \& 1); t += r1$$

这时若 $t \geq 2^{24}$ ，说明 $1.\text{frac} * 1.5 \geq 2$ ，结果要表示为 $1.\text{xxx} * 2^{(\text{exp} + 1)}$ 也就是要把 t 右移 1 位，而这可能导致再丢一个最低位，所以还要做一次最近偶舍入。

最后还要判断是否溢出，记

$$\text{newExp} = \text{exp} + \text{carry}$$

若 $\text{newExp} \geq 255$ ，说明结果超过最大有限浮点数，应返回无穷：

$$\text{return } (\text{sign} \ll 31) | 0x7F800000$$

而对于规格化一般情况，返回

$$\text{return } (\text{sign} \ll 31) | (\text{newExp} \ll 23) | \text{newFrac}$$

```
unsigned floatScaleThreeHalves(unsigned uf)
{
    unsigned sign = uf >> 31;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFF;
    if (exp == 0xFF) return uf;
    if (exp == 0)
    {
        unsigned t = frac + (frac >> 1);
        unsigned r = (frac & 1u) & (t & 1u);
        t += r;
        return (sign << 31) | t;
    }
    unsigned a = frac | 0x800000u;
    unsigned t = a + (a >> 1);
    unsigned r1 = (a & 1u) & (t & 1u);
    t += r1;
    unsigned carry = t >> 24;
    if (carry)
    {
        unsigned keep = t >> 1;
        unsigned r2 = (t & 1u) & (keep & 1u);
        t = keep + r2;
    }
    unsigned newExp = exp + carry;
    if (newExp >= 0xFF) return (sign << 31) | 0x7F800000u;
    unsigned newFrac = t & 0x7FFFFFu;
    return (sign << 31) | (newExp << 23) | newFrac;
}
```

P16 floatRoundEven

题目要求把浮点值舍到最近整数；正好一半时向偶数舍；若舍成 0 则需保留输入符号；NaN/Inf 原样返回。按照阶码分类（先转为指数 $e = \text{exp} - 127$ ）

对于 NaN/Infif(exp == 0xFF)

Return uf

对于 $e < -1$ 此时绝对值小于 0.5，应舍成带符号 0：

$\text{return sign} \ll 31$

对于 $e == -1$ 这表示区间 $[0.5, 1)$ ，如果正好 0.5 则向 0 舍；只要 $\text{frac} \neq 0$ ，值就严格大于 0.5，必须进到 1。

对于 $e \geq 23$ 小数部分已经不存在，值本身就是整数，直接返回。

对于 $0 \leq e \leq 22$ ，用 sig 表示补上隐藏位的 uf ，那么 sig 的低 $\text{shift} = 23 - e$ 位即小数部分。

$\text{value} = \text{sig} / 2^{\text{shift}}$

构造

$\text{keepMask} = \sim((1u \ll \text{shift}) - 1u)$

用于保留 frac 里不会被丢掉、也不会因为舍入发生进位前的那部分

$\text{keep} = \text{frac} \& \text{keepMask}$

接下来判断舍入，用

$\text{drop} = \text{frac} \& \sim \text{keepMask}$

表示小数部分分子（真正小数值 $= \text{drop} / 2^{\text{shift}}$ ）判断节点

$\text{half} = 1u \ll (\text{shift} - 1)$

若 $\text{drop} > \text{half}$ ：进位（ $\text{roundUp} = 1u$ ）；

若 $\text{drop} < \text{half}$ ：不进位（ $\text{roundUp} = 0$ ）；

若 $\text{drop} == \text{half}$ ：向偶数舍（ $\text{roundUp} = (\text{sig} \gg \text{shift}) \& 1u$ ）；

把进位加到保留部分的最低位

$\text{unsigned newFrac} = \text{keep} + (\text{roundUp} \ll \text{shift})$

而这一步可能产生进位，用

$\text{unsigned carry} = \text{newFrac} \gg 23$

判断是否进位到隐藏位，最终阶码为

$\text{newExp} = \text{exp} + \text{carry}$

```
unsigned floatRoundEven(unsigned uf)
{
    unsigned sign = uf >> 31;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFFFF;
    if (exp == 0xFF) return uf;
    int e = exp;
    e = e + (~127 + 1);
    if (e < -1)
    {
        return sign << 31;
    }
    if (e == -1)
    {
        if (frac)
        {
            return (sign << 31) | 0x3F800000u;
        }
        return sign << 31;
    }
    if (e >= 23) return uf;
    int shift = 23 - e;
    unsigned sig = frac | 0x800000u;
    unsigned keepMask = ~((1u << shift) - 1u);
    unsigned keep = frac & keepMask;
    unsigned drop = frac & (~keepMask);
    unsigned half = 1u << (shift - 1);
    unsigned roundUp = 0;
    if (drop > half)
    {
        roundUp = 1u;
    } else if (drop == half)
    {
        roundUp = (sig >> shift) & 1u;
    }
    unsigned newFrac = keep + (roundUp << shift);
    unsigned carry = newFrac >> 23;
    newFrac &= 0x7FFFFFFFu;
    unsigned newExp = exp + carry;
    return (sign << 31) | (newExp << 23) | newFrac;
}
```

P17 float_i2f

题目要求把 int 转成 IEEE754 单精度。对应特殊情况 0 直接返回即可。

对于一般情况，用

$$\text{mask} = x \gg 31$$

取符号；用

$$\text{absX} = (x \wedge \text{mask}) + (\sim \text{mask} + 1)$$

取绝对值（对于 $x = \text{INT_MIN}$ 也成立）

通过 while 循环寻找最高位，其下标记为 bits，从而规格化指数：

$$\text{exp} = \text{bits} + 127$$

接下来分情况构造尾数规格化数形式：

若 $\text{bits} \leq 23$ ，有效数字不足 24 位，左移补齐即可；

若 $\text{bits} > 23$ ，要右移丢掉低位，并做最近偶舍入（与上一条类似）

```
unsigned float_i2f(int x)
{
    unsigned mask;
    unsigned absX;
    unsigned exp;
    unsigned frac;
    unsigned v;
    int bits;
    if (x == 0) return 0;
    mask = x >> 31;
    absX = (x ^ mask) + (~mask + 1);
    v = absX;
    bits = 0;
    while (v >= 1) bits++;
    exp = bits + 127;
    if (bits > 23)
    {
        int shift = bits - 23;
        unsigned m = (1u << shift) - 1u;
        unsigned roundBits = absX & m;
        unsigned half = 1u << (shift - 1);
        frac = (absX >> shift) & 0x7FFFFFFu;
        if (roundBits > half || (roundBits == half && (frac & 1u)))
        {
            frac += 1u;
            if (frac >> 23)
            {
                exp += 1u;
                frac &= 0x7FFFFFFu;
            }
        }
    }
    else
    {
        int shift = 23 - bits;
        frac = (absX << shift) & 0x7FFFFFFu;
    }
    return (mask << 31) | (exp << 23) | frac;
}
```

P18 bitCount

题目本质数 x 的二进制中有多少个 1，即汉明重量。如果只是一位一位数那么操作数会超。

所以可以把 32 位分成小组，小组内先计数，再逐级合并。

先构造掩码（分别每 2/4/8 位一组）

$$m1 = 0x55555555$$

$$m2 = 0x33333333$$

$$m4 = 0x0F0F0F0F$$

然后通过

$$x = (x \& m1) + ((x \gg 1) \& m1)$$

把每 2 位中两个 bit 的 1 的个数存进这 2 位中。接着是 4 位和 8 位：

$$x = (x \& m2) + ((x \gg 2) \& m2)$$

$$x = (x \& m4) + ((x \gg 4) \& m4)$$

最后用

```
x = x + (x >> 8)
x = x + (x >> 16)
```

把各字节计数合并成总数。

```
int bitCount(int x)
{
    int m1 = 0x55;
    int m2 = 0x33;
    int m4 = 0x0F;
    m1 = m1 | (m1 << 8) | (m1 << 16) | (m1 << 24);
    m2 = m2 | (m2 << 8) | (m2 << 16) | (m2 << 24);
    m4 = m4 | (m4 << 8) | (m4 << 16) | (m4 << 24);
    x = (x & m1) + ((x >> 1) & m1);
    x = (x & m2) + ((x >> 2) & m2);
    x = (x & m4) + ((x >> 4) & m4);
    x = x + (x >> 8);
    x = x + (x >> 16);
    return x & 0x7F;
}
```

P19bitReverse

题目要把 32 位整数完全倒序，同样采用分治——依次交换相邻 1/2/4/8/16 位。

每一轮的形式都是：

$$x = ((x \gg k) \& m) | ((x \& m) \ll k)$$

即把被掩码选中的块右移，同时把没被掩码选中的块左移，再合并。

```
int bitReverse(int x)
{
    int m16 = 0xFF | (0xFF << 8);
    int m8 = m16 ^ (m16 << 8);
    int m4 = m8 ^ (m8 << 4);
    int m2 = m4 ^ (m4 << 2);
    int m1 = m2 ^ (m2 << 1);
    x = ((x >> 1) & m1) | ((x & m1) << 1);
    x = ((x >> 2) & m2) | ((x & m2) << 2);
    x = ((x >> 4) & m4) | ((x & m4) << 4);
    x = ((x >> 8) & m8) | ((x & m8) << 8);
    x = ((x >> 16) & m16) | (x << 16);
    return x;
}
```

三、检查和测试

(1) 检查运算符和操作数

```
● hilbert@LAPTOP-4PUAMS9C:~/25303090064lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 16/24 operators
midpointTowardFirst: 25/32 operators
isBetweenEitherOrder: 31/48 operators
mul5Sat: 30/30 operators
classifyAdd3: 39/52 operators
floatScaleThreeHalves: 27/60 operators
floatRoundEven: 26/65 operators
float_i2f: 34/40 operators
bitCount: 35/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

(2) 测试正确性

```
● hilbert@LAPTOP-4PUAMS9C:~/25303090064lab1$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

四、实验总结

(1) 整数题不能写大常量，所以统一用 0x55, 0x33, 0x0F, 0xFF 等小常量配合 $\ll$ 和 $|/\wedge$ 拼装大掩码。

(2) 使用符号掩码做无分支选择 ($x \gg 31$)

mask = -1 选 a, mask = 0 得 0:

mask & a

相反:

$\sim$ mask & b

(3) 用补码关系代替禁用的-

(4) 所有取负都写成:

$\sim x + 1$

从而减法可以写成:

$x + (\sim y + 1)$

(5) 溢出题不要先算总和，把可能溢出的中间结果拆成半值、差值符号或溢出方向，避免依赖回绕后的错误符号。

(6) 浮点题全部拆成 sign/exp/frac

规格化数一定要补隐藏位:

sig = frac | 0x800000

(7) 右移必须区分算术/逻辑，需要逻辑右移时，不能裸用 $\gg$ ，必须用掩码清符号扩展。